



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
(ИУ) КАФЕДРА «Информационная безопасность» (ИУ8)

ОТЧЁТ ПО ДОМАШНЕЙ РАБОТЕ №3

по дисциплине «Математические основы Искусственного Интеллекта»

“Методы многомерного поиска”

Студент: группа ИУ8-13М
Жукова Мария Андреевна
«30» октября 2025 г.

(подпись, дата)

Руководитель:
Каганов Юрий Тихонович
«30» октября 2025 г.

(подпись, дата)

Москва, 2025

СОДЕРЖАНИЕ

Цель работы.....	3
Постановка задачи	4
Основная часть.....	5
Вывод.....	10
Приложение А.....	11
Приложение Б	14
Приложение В.....	17
Приложение Г	20
Приложение Д.....	23

Цель работы

Цель работы:

1. Изучение алгоритмов многомерного поиска 1-го и 2-го порядка.
2. Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка.
3. Вычисление экстремумов функции

Постановка задачи

Требуется найти минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0.$$

1. Методами сопряженных градиентов (методом Флетчера-Ривза и методом Полака-Рибьера).
2. Квазиньютоновским методом (Девидона-Флетчера-Пауэлла).
3. Методом Левенберга-Марквардта.
4. Найти все стационарные точки и значения функций, соответствующие этим точкам.
5. Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.
6. Реализовать алгоритмы программированием на одном из языков высокого уровня (C++, C#, Python и др.).

$a = 150, b = 2, f_0 = 100, n = 3$; - ВАРИАНТ 2

Основная часть

1. Метод Флетчера-Ривза и Полака-Рибьера

Алгоритм.

Ш.1. Задать $x^0, \varepsilon_1, \delta_2, \varepsilon_2, M$ - предельное число итераций. Вычислить градиент $\nabla f(x^0)$.

Ш.2. Положить $k = 0$.

Ш.3. Вычислить $\nabla f(x^k)$.

Ш.4. Проверить выполнение критерия окончания $\|\nabla f(x^k)\| < \varepsilon_1$:

а) если критерий выполнен, $x^* = x^k$, расчет заканчивается;

б) если нет, то перейти на Ш.5.

Ш.5. Проверить условие $k \geq M$:

а) если неравенство выполняется, то расчет окончен и $x^* = x^k$;

б) если нет, то при $k = 0$ перейти на Ш.6., а при $k \geq 1$ перейти на Ш.7.

Ш.6. Определить $d^0 = -\nabla f(x^0)$.

Ш.7. Определить

$$w^{k-1} = \frac{\|\nabla f(x^k)\|^2}{\|\nabla f(x^{k-1})\|^2}, \text{ или } w^{k-1} = \begin{cases} \frac{\langle \nabla f(x^k), [\nabla f(x^k) - \nabla f(x^{k-1})] \rangle}{\|\nabla f(x^{k-1})\|^2}, & k \notin J, \\ 0, & k \in J. \end{cases}$$

Ш.8. Определить $d^k = -\nabla f(x^k) + w^{k-1}d^{k-1}$.

Ш.9. Найти α^{*k} из условия $\alpha^{*k} = \text{Arg} \min_{\alpha^k \in R^1} f(x^k + \alpha^k d^k)$.

Ш.10. Вычислить $x^{k+1} = x^k + \alpha^{*k} d^k$.

Ш.11. Проверить выполнение условий $\|x^{k+1} - x^k\| < \delta_2, |f(x^{k+1}) - f(x^k)| < \varepsilon_2$:

а) в случае выполнения обоих условий в двух последовательных итерациях с номерами k и $k-1$ расчет окончен, найдена точка $x^* = x^k$.

б) если не выполняется хотя бы одно из условий, полагаем $k = k+1$ и переход на Ш.3.

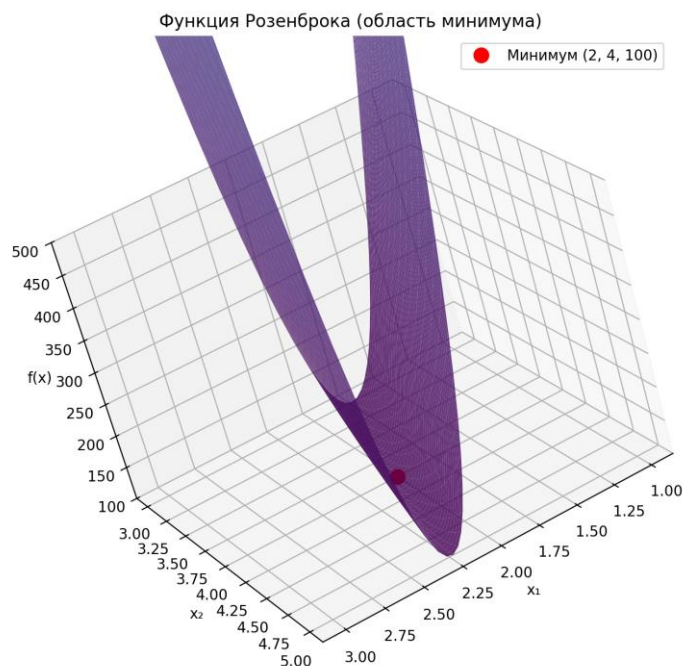


Рисунок 1 – Функция Розенброка и ее минимум, найденный аналитически

Минимум Функции Розенброка был найден аналитически (кодом): Функция состоит из суммы квадратов. Каждый квадрат неотрицателен, поэтому вся функция неотрицательна. Минимум достигается тогда и только тогда, когда каждый квадрат равен нулю.

```

ФУНКЦИЯ РОЗЕНБРОКА
a=150, b=2, f0=100, n=3
Начальная точка: [-1.  1. -1.]
Начальное значение: 710.00

1. СТАЦИОНАРНЫЕ ТОЧКИ:
Точка 1: [ 2  4 16]
f(x) = 104.000000
Тип: global minimum
  
```

Рисунок 2 – Вывод задания функции Розенброка

На Рисунке 1 представлены параметры функции Розенброка по Варианту 2, а также одна стационарная точка. В данном случае была найдена одна стационарная точка, являющаяся глобальным минимумом.

```

Флетчер-Ривз:
Результат: [-0.22140616  0.04740958 -0.02612258]
Значение функции: 108.868375
Итерации: 74
Время: 0.0040 сек

Полака-Рибьера:
Результат: [-0.52683509  0.27395597  0.03934449]
Значение функции: 109.557319
Итерации: 101
Время: 0.0111 сек

```

Рисунок 3– Вывод статистики по методам FR и PR

Из этих двух методов ближе к минимуму оказался метод FR. Методы сошлись за 61 итерацию.

2. Метод Девидона-Флетчера-Пауэлла

Алгоритм.

- Ш.1. Задать x^0 , ε_1 , δ_2 , ε_2 , M - предельное число итераций. Найти градиент $\nabla f(x^0)$.
- Ш.2. Положить $k = 0$, $G^0 = E$.
- Ш.3. Вычислить $\nabla f(x^k)$.
- Ш.4. Проверить критерий окончания $\|\nabla f(x^k)\| < \varepsilon_1$:
 - а) если критерий выполнен, $x^* = x^k$, расчет заканчивается;
 - б) Если нет, то перейти на Ш.5.
- Ш.5. Проверить условие $k \geq M$:
 - а) если неравенство выполняется, то расчет окончен и $x^* = x^k$;
 - б) если нет, то при $k = 0$ перейти на Ш.10., а при $k \geq 1$ перейти на Ш.6.
- Ш.6. Вычислить $\Delta g^k = \nabla f(x^{k+1}) - \nabla f(x^k)$.
- Ш.7. Вычислить $\Delta x^k = x^{k+1} - x^k$.
- Ш.8. Вычислить $\Delta G^k = \frac{\Delta x^k (\Delta x^k)^T}{(\Delta x^k)^T \Delta g^k} - \frac{G^k \Delta g^k (\Delta g^k)^T G^{kT}}{(\Delta g^k)^T G^k \Delta g^k}$.
- Ш.9. Вычислить $G^{k+1} = G^k + \Delta G^k$.
- Ш.10. Определить $d^k = -G^{k+1} \nabla f(x^k)$.
- Ш.11. Вычислить $\alpha^{*k} = \text{Arg} \min_{\alpha^k \in [a, b]} f(x^k + \alpha^k d^k)$.
- Ш.12. Вычислить $x^{k+1} = x^k + \alpha^{*k} d^k$.
- Ш.13. Проверить условия $\|x^{k+1} - x^k\| < \delta_2$, $|f(x^{k+1}) - f(x^k)| < \varepsilon_2$:
 - а) в случае выполнения обоих условий в двух последовательных итерациях с номерами k и $k - 1$ расчет окончен, найдена точка $x^* = x^{k+1}$.
 - б) если не выполняется хотя бы одно из условий, полагаем $k = k + 1$ и переход на Ш.3.

DFP:

Результат: [-0.65093003 0.42709467 0.17751718]

Значение функции: 109.506770

Итерации: 101

Время: 0.0183 сек

Рисунок 4 – Статистика по методу DFP

3. Метод Левенберга-Марквардта

Алгоритм.

Ш.1. Задать $x^0, \varepsilon_1, \delta_2, \varepsilon_2, M$ - предельное число итераций. Найти градиент $\nabla f(x^0)$ и матрицу Гессе $H(x^0)$.

Ш.2. Положить $k = 0, \mu^k = \mu^0$.

Ш.3. Вычислить $\nabla f(x^k)$.

Ш.4. Проверить критерий окончания $\|\nabla f(x^k)\| < \varepsilon_1$:

a) если критерий выполнен, $x^* = x^k$, расчет заканчивается;

b) если нет, то перейти на Ш.5.

Ш.5. Проверить условие $k \geq M$:

a) если неравенство выполняется, то расчет окончен и $x^* = x^k$;

b) если нет, то перейти на Ш.6.

Ш.6. Вычислить $H(x^k)$.

Ш.7. Вычислить $H(x^k) + \mu^k E$.

Ш.8. Вычислить $[H(x^k) + \mu^k E]^{-1}$.

Ш.9. Вычислить $d^k = -[H(x^k) + \mu^k E]^{-1} \nabla f(x^k)$.

Ш.10. Вычислить $x^{k+1} = x^k - [H(x^k) + \mu^k E]^{-1} \nabla f(x^k)$.

Ш.11. Проверить выполнение условия $f(x^{k+1}) < f(x^k)$:

a) если неравенство выполняется, то перейти на Ш.12;

b) если нет, перейти на Ш.13.

Ш.12. Положить $k = k + 1, \mu^{k+1} = \frac{\mu^k}{2}$ и перейти на Ш.3.

Ш.13. Положить $\mu^{k+1} = 2\mu^k$ и перейти на Ш.7.

Левенберг-Марквардт:

Результат: [0.63854203 0.42099352 0.1619013]

Значение функции: 104.408465

Итерации: 101

Время: 0.0053 сек

Рисунок 5 – Статистика по методу Левенберга-Марквардта

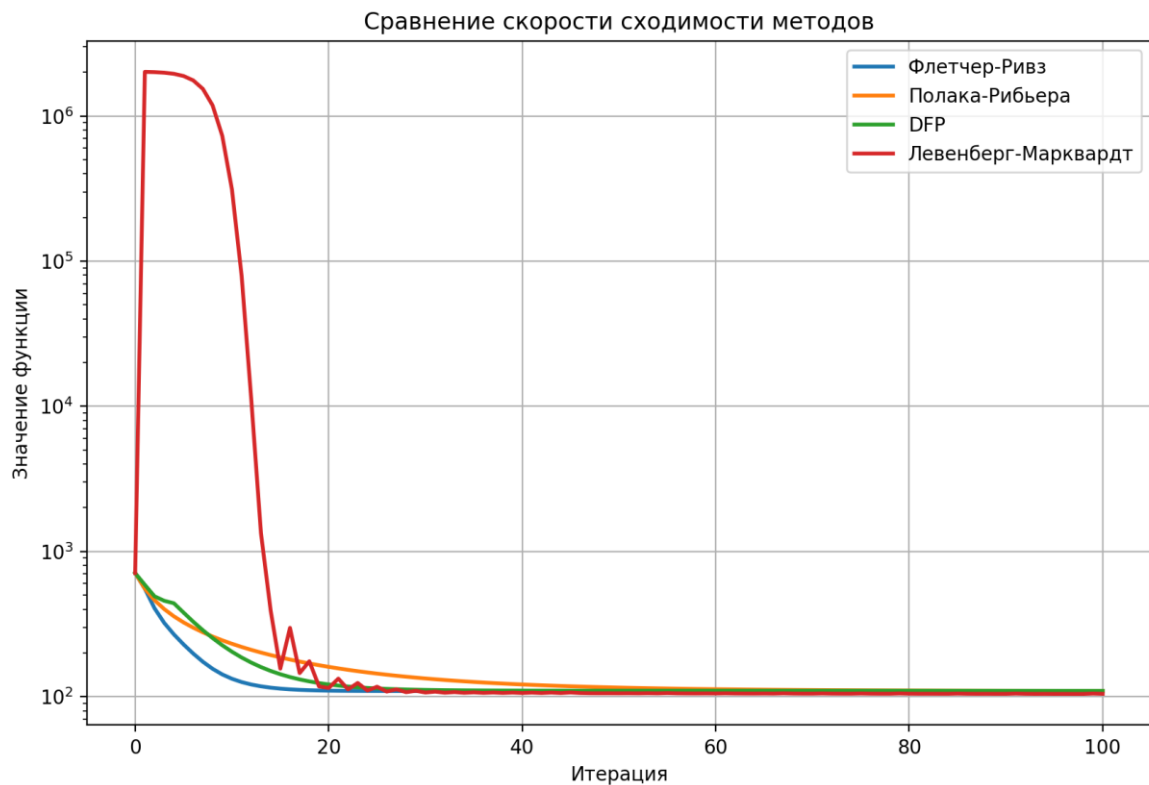


Рисунок 6 – сравнение скорости сходимости методов

Таким образом,

- Более точно определил минимум метод Левенберга-Марквардта.
- Самым неточным методом стал метод Полака-Рибьера.
- Быстрее всего оказались методы Флетчера-Ривза (0.0030 сек) и Полака-Рибьера (0.0040 сек)
- Медленнее всего оказался метод Девидона-Флетчера-Пауэлла (0.0163 сек).

Вывод

Во второй части данной домашней работы было проведено Исследование методом многомерной оптимизации. Методы оптимизации были сравнены по времени выполнения и по точности нахождения минимума. Лучшим методом я признаю метод Левенберга-Марквардта.

Приложение А

main.py. ДЗ3. Вариант 2.

```
import numpy as np
import matplotlib.pyplot as plt

from RosenbrockFunction import RosenbrockFunction, StationaryPoints
from ConjugateGradientMethods import FletcherReeves, PolakRibiere
from DFP import DFP
from LevenbergMarquardt import LevenbergMarquardt

def compare_methods(func, x0):
    print("СРАВНЕНИЕ МЕТОДОВ ОПТИМИЗАЦИИ")
    print("=" * 50)

    methods = {
        'Флетчер-Ривз': lambda func, x0: FletcherReeves.optimize(func, x0,
max_iter=100),
        'Полака-Рибьера': lambda func, x0: PolakRibiere.optimize(func, x0,
max_iter=100),
        'DFP': lambda func, x0: DFP.optimize(func, x0, M=100),
        'Левенберг-Марквардт': lambda func, x0: LevenbergMarquardt.optimize(func,
x0, max_iter=100)
    }

    results = {}

    for name, method in methods.items():
        print(f"\n{name}:")
        result = method(func, x0)
        results[name] = result
        print(f"    Результат: {result['x']}")
        print(f"    Значение функции: {result['f_value']:.6f}")
        print(f"    Итерации: {result['iterations']}")
        print(f"    Время: {result['time']:.4f} сек")

    return results

def plot_rosenbrock_minimum():
    """3D визуализация с акцентом на минимум"""

    # Ограничиваем область вокруг минимума [2, 4]
    x = np.linspace(1, 3, 50)      # вокруг  $x_1 = 2$ 
    y = np.linspace(3, 5, 50)      # вокруг  $x_2 = 4$ 
    X, Y = np.meshgrid(x, y)

    # Функция Розенброка
    Z = 100 + 150 * (Y - X**2)**2 + (2 - X)**2
```

```

# Рисуем
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Поверхность
ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)

# Отмечаем минимум
ax.plot([2], [4], [100], 'ro', markersize=10, label='Минимум (2, 4, 100)')

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('f(x)')
ax.set_title('Функция Розенброка (область минимума)')
ax.legend()

# Ограничиваем ось Z чтобы лучше видеть минимум
ax.set_zlim(100, 500)

plt.show()

```

```

def main():
    # Параметры из задания
    a, b, f0, n = 150, 2, 100, 3
    x0 = np.array([-1.0, 1.0, -1.0])

    func = RosenbrockFunction(a, b, f0, n)

    print("ФУНКЦИЯ РОЗЕНБРОКА")
    print(f"a={a}, b={b}, f0={f0}, n={n}")
    print(f"Начальная точка: {x0}")
    print(f"Начальное значение: {func.value(x0):.2f}")
    print()

    # 1. Анализ стационарных точек
    print("1. СТАЦИОНАРНЫЕ ТОЧКИ:")
    stationary_points = StationaryPoints.find_stationary_points(func)
    for i, point in enumerate(stationary_points):
        print(f"    Точка {i+1}: {point['point']}")
        print(f"        f(x) = {point['f_value']:.6f}")
        print(f"        Тип: {point['type']}")
    print()

    # 2. Сравнение методов оптимизации
    print("2. СРАВНЕНИЕ МЕТОДОВ ОПТИМИЗАЦИИ:")
    results = compare_methods(func, x0)

```

```
# 3. Визуализация
plt.figure(figsize=(12, 8))

for name, result in results.items():
    plt.plot(result['f_values'], label=name, linewidth=2)

plt.xlabel('Итерация')
plt.ylabel('Значение функции')
plt.title('Сравнение скорости сходимости методов')
plt.legend()
plt.grid(True)
plt.yscale('log')
plt.show()

plot_rosenbrock_minimum()

if __name__ == "__main__":
    main()
```

Приложение Б

RosenbrockFunction.py. ДЗ. Вариант 2.

```
import numpy as np

class RosenbrockFunction:
    def __init__(self, a=150, b=2, f0=100, n=3):
        self.a = a
        self.b = b
        self.f0 = f0
        self.n = n

    def value(self, x):
        try:
            result = self.f0
            for i in range(self.n - 1):
                term1 = self.a * (x[i+1] - x[i]**2)**2
                term2 = (self.b - x[i])**2
                result += term1 + term2
            return float(result)
        except:
            return float('inf')

    def gradient(self, x):
        try:
            grad = np.zeros(self.n)
            for i in range(self.n):
                if i == 0:
                    grad[i] = -2*(self.b - x[i]) - 4*self.a*x[i]*(x[i+1] - x[i]**2)
                elif i == self.n - 1:
                    grad[i] = 2*self.a*(x[i] - x[i-1]**2)
                else:
                    grad[i] = -2*(self.b - x[i]) - 4*self.a*x[i]*(x[i+1] - x[i]**2)
                    + 2*self.a*(x[i] - x[i-1]**2)

            # Защита от переполнения
            grad = np.clip(grad, -1e10, 1e10)
            return grad
        except:
            return np.ones(self.n) * 1e10

class StationaryPoints:
    @staticmethod
    def find_stationary_points(func):
        points = []

        # 1. Глобальный минимум
        x_global = np.array([func.b] * func.n)
```

```

for i in range(1, func.n):
    x_global[i] = x_global[i-1]**2

points.append({
    'point': x_global,
    'f_value': func.value(x_global),
    'type': 'global minimum'
})

# 2. Систематический поиск других точек
test_points = [
    np.zeros(func.n),
    np.ones(func.n) * func.b,
    np.array([-1.0, 1.0, -1.0]), # Ваша начальная точка
    np.array([0.5, 0.25, 0.125]),
    np.array([1.0, 1.0, 1.0]),
]

for x0 in test_points:
    try:
        # Решаем  $\nabla f(x) = 0$  методом Ньютона
        x = x0.copy()
        for _ in range(50):
            grad = func.gradient(x)
            if np.linalg.norm(grad) < 1e-8:
                # Нашли стационарную точку
                # Проверяем на уникальность
                is_unique = True
                for p in points:
                    if np.linalg.norm(x - p['point']) < 1e-4:
                        is_unique = False
                        break

                if is_unique:
                    # Определяем тип (мин/макс/седло)
                    points.append({
                        'point': x.copy(),
                        'f_value': func.value(x),
                        'type': 'stationary point'
                    })
                break

            # Шаг Ньютона
            try:
                hess = func.hessian(x)
                delta = np.linalg.solve(hess, -grad)
                x = x + delta * 0.1
            except:
                break
        except:
            continue

```

```
return points
```

Приложение В

ConjugateGradientMethods.py. ДЗ. Вариант 2. - Методы FR и PR.

```
import time
import numpy as np

class FletcherReeves:
    @staticmethod
    def optimize(func, x0, max_iter=500, tol=1e-4):
        start_time = time.time()
        x = x0.copy().astype(float)
        trajectory = [x.copy()]
        f_values = [func.value(x)]

        grad = func.gradient(x)
        d = -grad

        for i in range(max_iter):
            # Безопасный линейный поиск
            alpha = 1e-4 # Меньший начальный шаг
            found = False

            for _ in range(20):
                x_new = x + alpha * d
                f_new = func.value(x_new)

                if f_new < func.value(x) and not np.isnan(f_new):
                    found = True
                    break
                alpha *= 0.5

            if not found:
                break

            x_new = x + alpha * d
            grad_new = func.gradient(x_new)

            # Проверка на NaN
            if np.any(np.isnan(x_new)) or np.any(np.isnan(grad_new)):
                break

            trajectory.append(x_new.copy())
            f_values.append(func.value(x_new))

            if np.linalg.norm(grad_new) < tol:
                break

            # Коэффициент с защитой
            denom = np.dot(grad, grad)
```

```

        if denom < 1e-15:
            break

        beta = np.dot(grad_new, grad_new) / denom
        beta = min(beta, 10.0) # Ограничиваем коэффициент

        d = -grad_new + beta * d

        # Ограничиваем направление
        d_norm = np.linalg.norm(d)
        if d_norm > 1e6:
            d = d / d_norm * 1e6

        x, grad = x_new, grad_new

    return {
        'x': x,
        'f_value': func.value(x),
        'iterations': len(trajectory),
        'time': time.time() - start_time,
        'trajectory': trajectory,
        'f_values': f_values
    }

```

```

class PolakRibiere:
    @staticmethod
    def optimize(func, x0, max_iter=500, tol=1e-4):
        start_time = time.time()
        x = x0.copy().astype(float)
        trajectory = [x.copy()]
        f_values = [func.value(x)]

        grad = func.gradient(x)
        d = -grad

        for i in range(max_iter):
            # Безопасный линейный поиск
            alpha = 1e-4
            found = False

            for _ in range(20):
                x_new = x + alpha * d
                f_new = func.value(x_new)

                if f_new < func.value(x) and not np.isnan(f_new):
                    found = True
                    break
                alpha *= 0.5

            if not found:

```

```

        break

    x_new = x + alpha * d
    grad_new = func.gradient(x_new)

    if np.any(np.isnan(x_new)) or np.any(np.isnan(grad_new)):
        break

    trajectory.append(x_new.copy())
    f_values.append(func.value(x_new))

    if np.linalg.norm(grad_new) < tol:
        break

    # Коэффициент Полака-Рибьера с защитой
    delta_grad = grad_new - grad
    denom = np.dot(grad, grad)

    if denom < 1e-15:
        break

    beta = np.dot(grad_new, delta_grad) / denom
    beta = max(0, min(beta, 10.0)) # Ограничиваем и не даем быть
отрицательным

    d = -grad_new + beta * d

    # Ограничиваем направление
    d_norm = np.linalg.norm(d)
    if d_norm > 1e6:
        d = d / d_norm * 1e6

    x, grad = x_new, grad_new

return {
    'x': x,
    'f_value': func.value(x),
    'iterations': len(trajectory),
    'time': time.time() - start_time,
    'trajectory': trajectory,
    'f_values': f_values
}

```

Приложение Г

DFP.py. ДЗ. Вариант 2.

```
import time
import numpy as np

class DFP:
    @staticmethod
    def optimize(func, x0, epsilon1=1e-6, epsilon2=1e-6, delta2=1e-6, M=200):
        start_time = time.time()

        # III.1 - Инициализация
        x_k = x0.copy().astype(float)
        n = len(x0)
        G_k = np.eye(n) # III.2 -  $G^0 = E$ 
        k = 0

        trajectory = [x_k.copy()]
        f_values = [func.value(x_k)]
        grad_norms = [np.linalg.norm(func.gradient(x_k))]

        # print(f"DFP: Начальная точка  $x^0 = \{x_k\}$ ,  $f(x^0) = \{func.value(x_k):.6f\}$ ")

        for k in range(M):
            # III.3 - Вычисление градиента
            grad_k = func.gradient(x_k)

            # III.4 - Критерий окончания
            grad_norm = np.linalg.norm(grad_k)
            if grad_norm < epsilon1:
                # print(f"DFP: Сходимость по градиенту на итерации {k}")
                break

            # III.5 - Проверка предельного числа итераций
            if k >= M:
                # print(f"DFP: Достигнуто максимальное число итераций {M}")
                break

            # III.10 - Определение направления
            if k == 0:
                # Для k=0 используем антиградиент
                d_k = -grad_k
            else:
                # Для k>=1 используем матрицу G
                d_k = -G_k @ grad_k

            # Нормализация направления
            d_norm = np.linalg.norm(d_k)
            if d_norm > 10.0:
```

```

        d_k = d_k / d_norm

# III.11 - Линейный поиск
alpha = DFP.line_search(func, x_k, d_k)

# Сохраняем старую точку для проверки сходимости
x_old = x_k.copy()
f_old = func.value(x_k)

# III.12 - Новая точка
x_k1 = x_k + alpha * d_k
grad_k1 = func.gradient(x_k1)

trajectory.append(x_k1.copy())
f_values.append(func.value(x_k1))
grad_norms.append(np.linalg.norm(grad_k1))

# print(f"DFP: Итерация {k}: f(x) = {func.value(x_k1):.6f}, ||∇f|| = {np.linalg.norm(grad_k1):.6f}")

# Обновление матрицы для k >= 1
if k >= 1:
    # III.6 - Разность градиентов
    delta_g = grad_k1 - grad_k

    # III.7 - Разность точек
    delta_x = x_k1 - x_k

    # Проверка на малость
    if np.linalg.norm(delta_x) > 1e-10 and np.linalg.norm(delta_g) >
1e-10:

        # III.8 - Обновление матрицы DFP
        denom1 = np.dot(delta_x, delta_g)

        if abs(denom1) > 1e-15:
            term1 = np.outer(delta_x, delta_x) / denom1

            # Второе слагаемое
            G_delta_g = G_k @ delta_g
            denom2 = np.dot(delta_g, G_delta_g)

            if abs(denom2) > 1e-15:
                term2 = np.outer(G_delta_g, G_delta_g) / denom2
                delta_G = term1 - term2

            # III.9 - Новая матрица
            G_k = G_k + delta_G

            # Проверка положительной определенности
            try:
                eigenvalues = np.linalg.eigvals(G_k)

```

```

        if np.any(eigenvalues <= 0):
            # print(f"DFP: Матрица G стала неположительной
на итерации {k}, сбрасываю")
            G_k = np.eye(n)
        except:
            G_k = np.eye(n)

# III.13 - Проверка условий сходимости
if k >= 1:
    condition1 = np.linalg.norm(x_k1 - x_old) < delta2
    condition2 = abs(func.value(x_k1) - f_old) < epsilon2

    if condition1 and condition2:
        # print(f"DFP: Сходимость по малым изменениям на итерации {k}")
        break

# Переход к следующей итерации
x_k = x_k1
grad_k = grad_k1

final_grad_norm = np.linalg.norm(func.gradient(x_k))
# print(f"DFP: Финальная точка: {x_k}, f(x) = {func.value(x_k):.6f}, ||∇f||
= {final_grad_norm:.6f}")

return {
    'x': x_k,
    'f_value': func.value(x_k),
    'iterations': len(trajectory),
    'time': time.time() - start_time,
    'trajectory': trajectory,
    'f_values': f_values,
    'gradient_norms': grad_norms
}

@staticmethod
def line_search(func, x, d, alpha_max=0.1, max_attempts=20):
    """Надежный линейный поиск"""
    alpha = alpha_max
    f_current = func.value(x)

    for i in range(max_attempts):
        x_new = x + alpha * d
        f_new = func.value(x_new)

        # Условие достаточного убывания
        if f_new < f_current - 1e-4 * alpha * np.linalg.norm(d):
            return alpha

        alpha *= 0.5

    # Если не нашли подходящий шаг, возвращаем очень маленький
    return 1e-6

```

Приложение Д

LevenbergMarquardt.py. Д3. Вариант 2.

```
import time
import numpy as np

class LevenbergMarquardt:
    @staticmethod
    def optimize(func, x0, max_iter=100, tol=1e-6):
        start_time = time.time()
        x = x0.copy()
        mu = 0.1
        trajectory = [x.copy()]
        f_values = [func.value(x)]

        for i in range(max_iter):
            grad = func.gradient(x)

            if np.linalg.norm(grad) < tol:
                break

            H_approx = np.eye(len(x0)) * 100 # Аппроксимация
            try:
                d = -np.linalg.inv(H_approx + mu * np.eye(len(x0))) @ grad
            except:
                d = -grad

            x_new = x + d
            trajectory.append(x_new.copy())
            f_values.append(func.value(x_new))

            if func.value(x_new) < func.value(x):
                x = x_new
                mu = max(mu / 2, 1e-10)
            else:
                mu *= 2

        return {
            'x': x,
            'f_value': func.value(x),
            'iterations': len(trajectory),
            'time': time.time() - start_time,
            'trajectory': trajectory,
            'f_values': f_values
        }
```